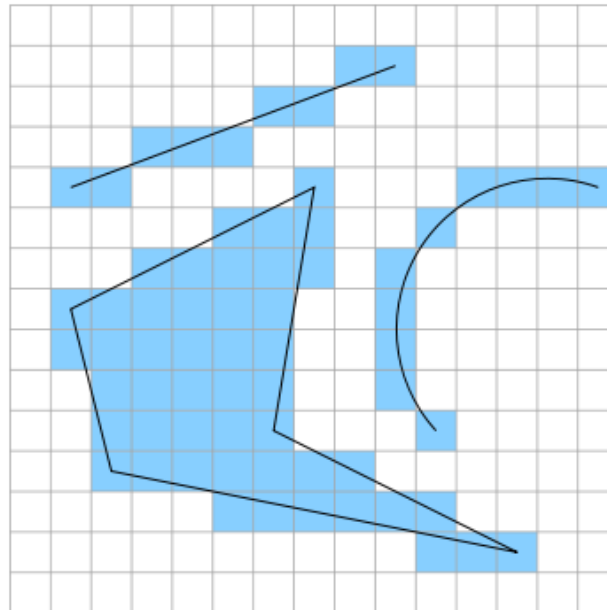
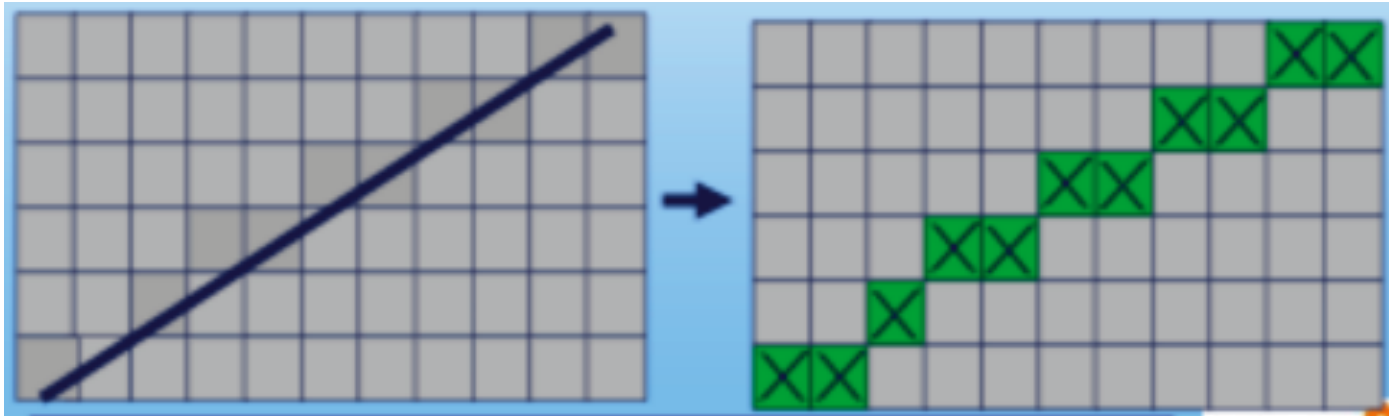


**4003-570/4005-761**  
**Computer Graphics 1**

**Drawing Lines and Circles**

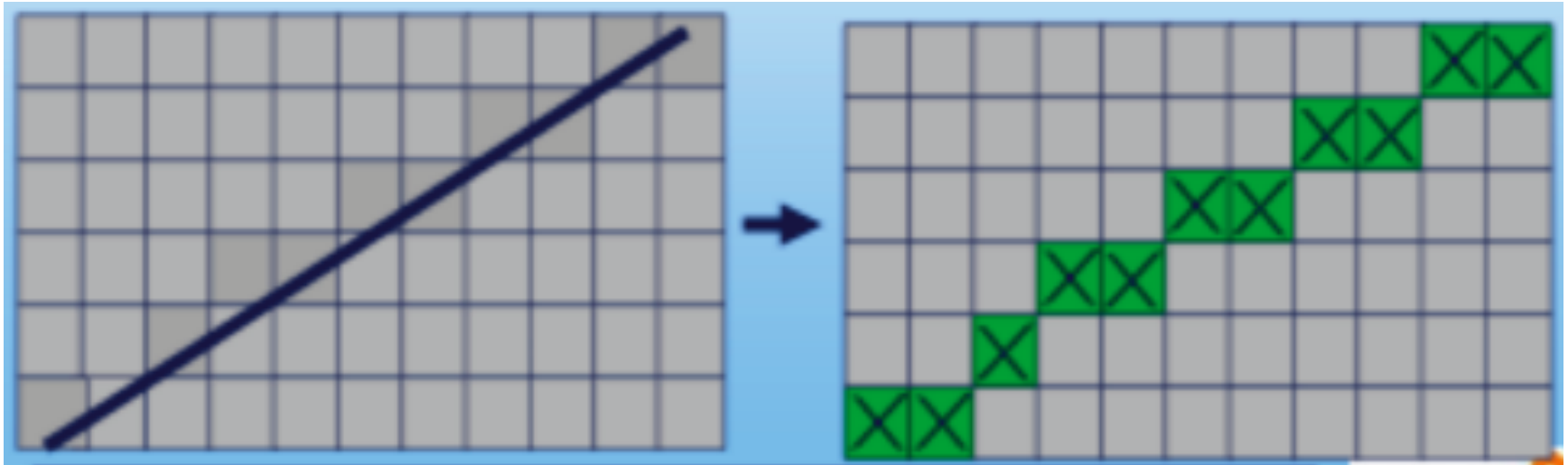
# Rasterization

- Turn screen space vertex coordinates into pixels that make up lines and polygons



# Rasterization

- Turn screen space vertex coordinates into pixels that make up lines and polygons

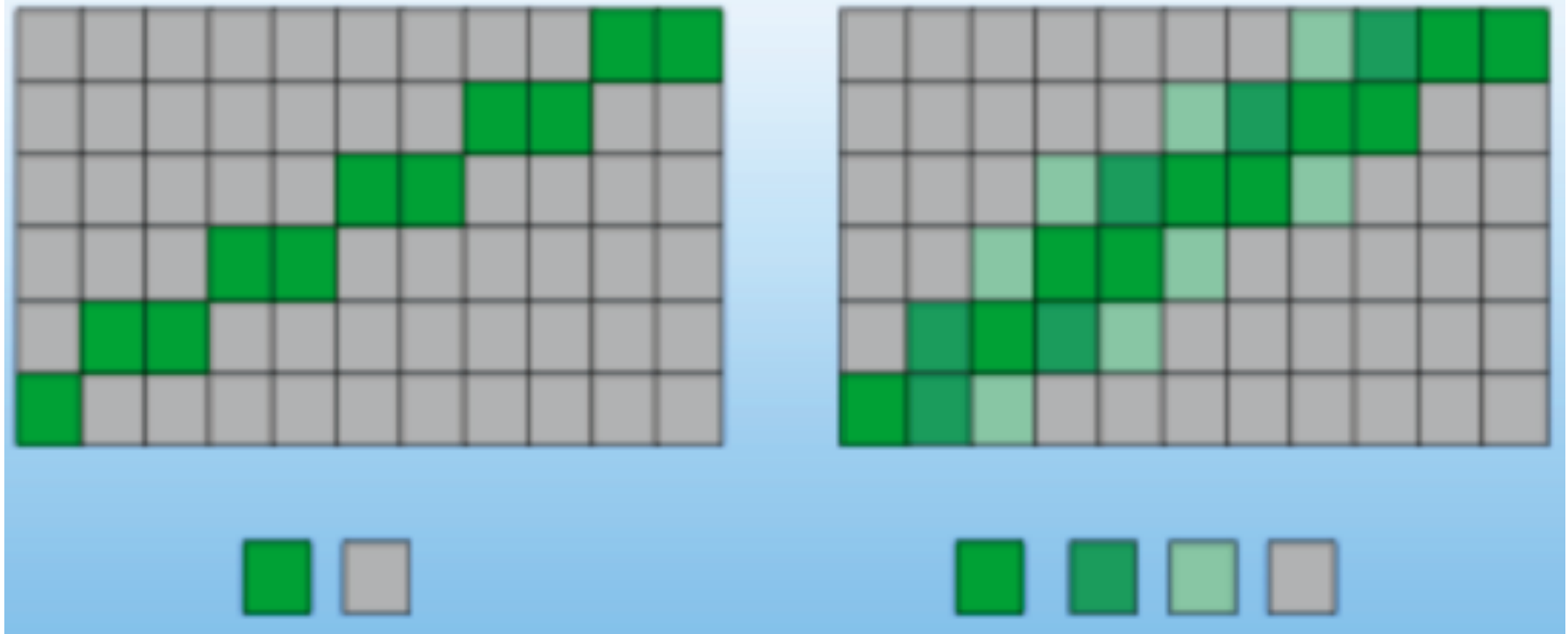


Continuous

Discrete

# Antialiasing

## Anti-Aliasing



# Drawing Lines

---

- Easy in OpenGL:

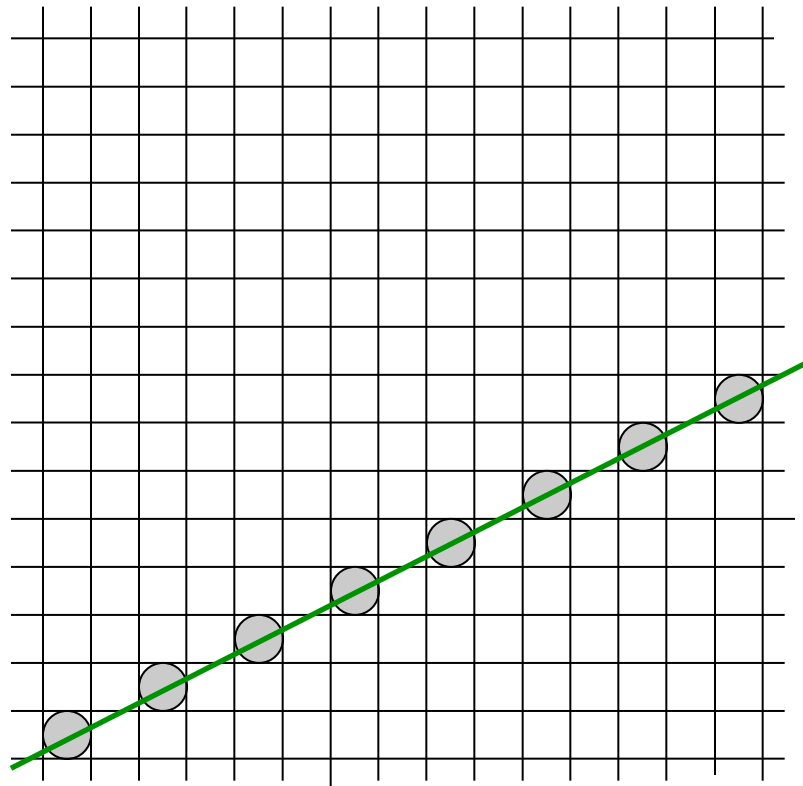
```
glBegin( GL_LINES );  
    glVertex2i( 10, 20 );  
    glVertex2i( 50, 100 );  
glEnd();
```

Note: this function has been deprecated/eliminated in newer versions of OpenGL

- Useful to know how it works under the hood
- What we want in a line drawing algorithm:
  - Fast
  - Constant/uniform width & brightness regardless of slope or location
  - As straight as possible
  - Equal-length lines have comparable lengths

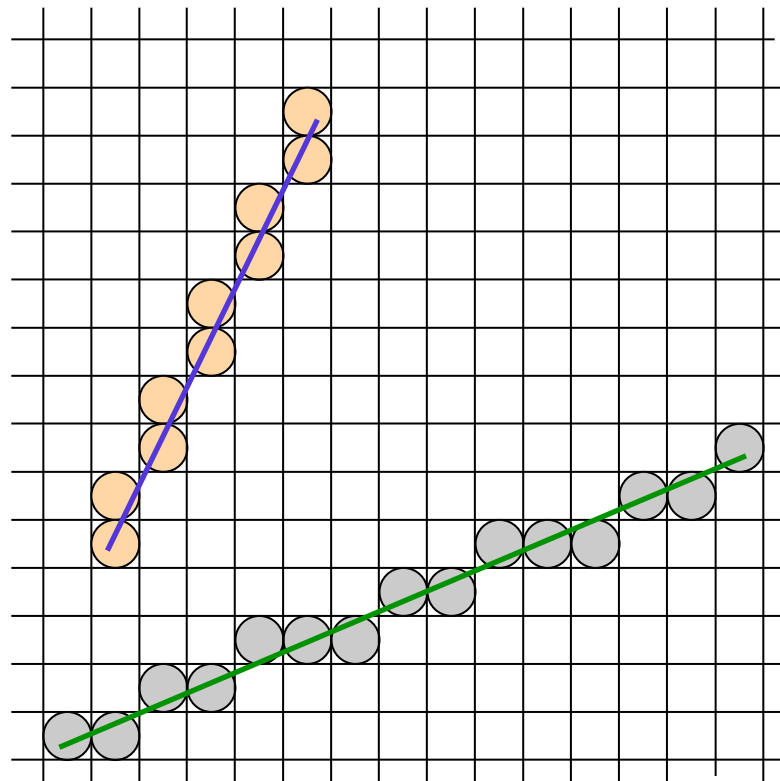
# Scan Converting Lines

- Could turn on all pixels mostly covered by the line
  - May leave gaps as illustrated here
  - Or may cause thick regions
- How to efficiently determine which pixels to color?



# Scan-Converting Lines

- Concept: compute coordinates of pixels that lie on or near an ideal, infinitely thin straight line imposed on a 2D raster grid
- For small slopes ( $-1 < m < 1$ ), illuminate one pixel per column; for large slopes, one pixel per row



# Slope-Intercept Equation

---

- Recall the standard equation for a line:

$$y = mx + B$$

- Could implement this directly:
  - $0 < m < 1$ ,  $\Delta x$  is 1; compute  $y$
  - $m > 1$ ,  $\Delta y$  is 1; compute  $x$
  - Round computed coordinate to nearest integer
- Negative slopes are handled similarly



# Slope-Intercept Equation

- Implementation, for a line  $p1$  to  $p2$  with slope  $0 < m < 1$ :

```
float m = (p2.y - p1.y) / (p2.x - p1.x);  
float B = p1.y - m * p1.x;
```

```
for( int x = p1.x; x <= p2.x; ++x ) {  
    setPixel( x, floor(m* x + B) );  
    y += m;  
}
```

- Problem: doing lots of floating point math
  - FP is significantly slower than integer math
  - Also suffers from roundoff error (causes precision problems)

# Digital Differential Analyzer

---

- Better idea: figure out how  $x$  and  $y$  change each iteration, and do the computation incrementally
- Called the *digital differential analyzer* (DDA)
  - Also known as the *basic incremental algorithm*
- Compute delta values for  $x$  and  $y$ 
  - Delta is the fractional change in that coordinate for each iteration
- Color in the pixel closest to  $(x,y)$  each iteration
- Number of iterations depends on the slope
  - Small positive: determined by  $\Delta x$
  - Large positive: determined by  $\Delta y$

# DDA Implementation

---

```
inline int round (const float a ) { return( (int) (a + 0.5) ); }

void lineDDA(int x0, int y0, int xEnd, int yEnd) {
    int dx = xEnd - x0, dy = yEnd - y0, steps, k;
    float xIncrement, yIncrement, x = x0, y = y0;

    if (fabs (dx) > fabs(dy)
        steps = fabs(dx);
    else
        steps = fabs(dy);

    xIncrement = (float) dx / (float) steps;
    yIncrement = (float) dy / (float) steps;

    setPixel( round(x), round(y) );
    for( k = 0; k < steps; k++ ) {
        x += xIncrement;
        y += yIncrement
        setPixel( round(x), round(y) );
    }
}
```

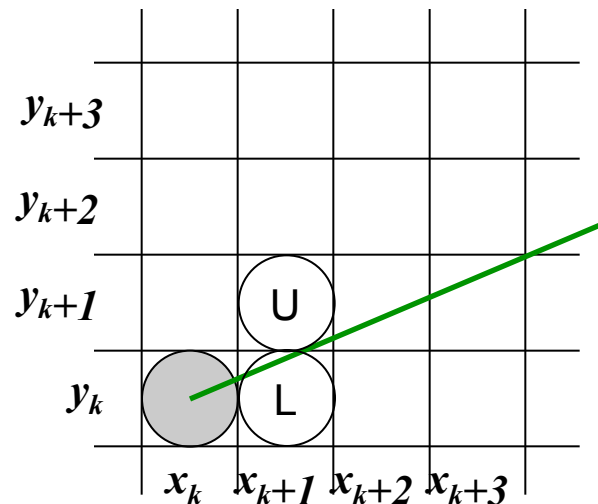
## DDA Summary

---

- Faster than direct implementation of slope/intercept equation
  - No FP multiplication each iteration
- Works for both small and large slopes
- However, still dealing with FP arithmetic
  - Over time, accumulate roundoff error
  - Can result in misplaced pixels in long line segments
- Can we replace FP arithmetic with integer?

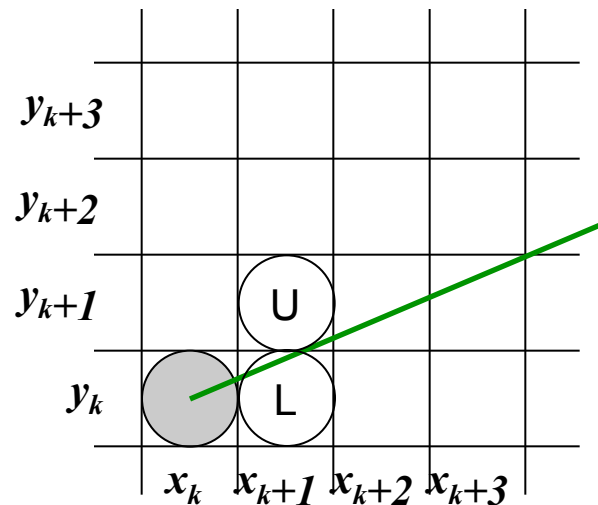
# Bresenham's Line Algorithm

- Jack Bresenham developed a better algorithm that uses only integer arithmetic
  - Called *Bresenham's Algorithm*
  - Designed for controlling a digital plotter
- Consider a line with small positive slope ( $0 < m < 1$ )



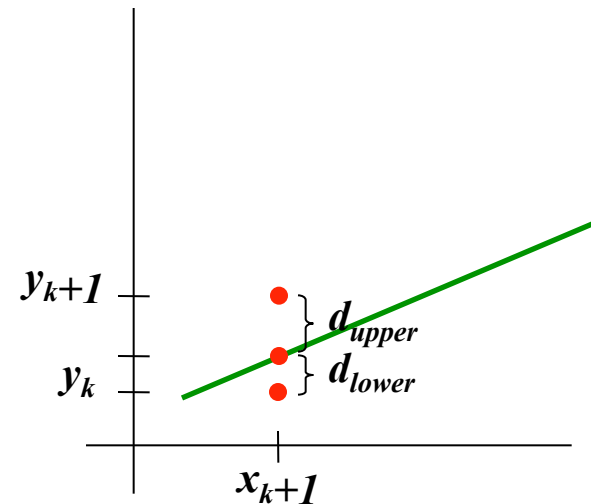
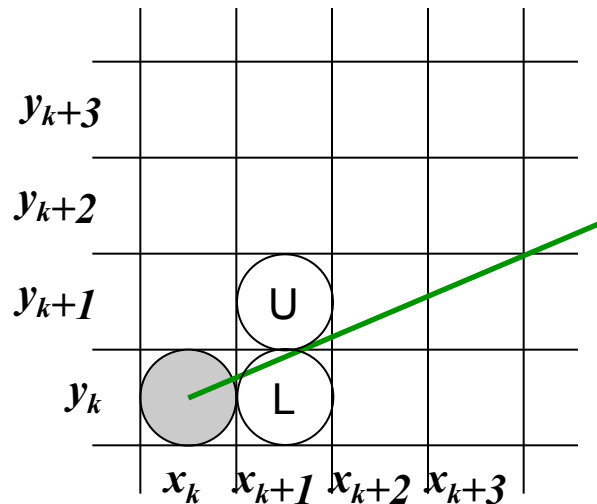
# Bresenham's Line Algorithm

- Given:
  - $0 < m < 1$
  - Pixel at  $(x_k, y_k)$  is the closest one to the actual line for  $x_k$
- The closest pixel for  $x_{k+1}$  must be one of two pixels:
  - Point L:  $(x_{k+1}, y_k)$
  - Point U:  $(x_{k+1}, y_k + 1)$



# Bresenham's Line Algorithm

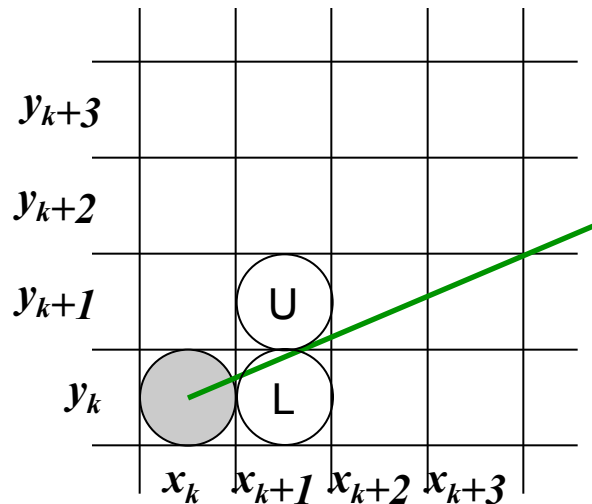
- The question: which (U or L) is closer to the actual line?
- The solution: compare the distances from the actual intersection point to U ( $d_{upper}$ ) and L ( $d_{lower}$ )
- We calculate this using the slope-intercept equation



# Bresenham's Line Algorithm

- The actual point:  $y = m(x_k + 1) + b$
- Points U and L:  $d_{lower} = y - y_k = m(x_k + 1) + b - y_k$   
 $d_{upper} = (y_k + 1) - y = y_k + 1 - m(x_k + 1) - b$
- Compare the distances via subtraction:

$$\begin{aligned}d_{lower} - d_{upper} &= (m(x_k + 1) + b - y_k) - (y_k + 1 - m(x_k + 1) - b) \\&= 2m(x_k + 1) - 2y_k + 2b - 1\end{aligned}$$





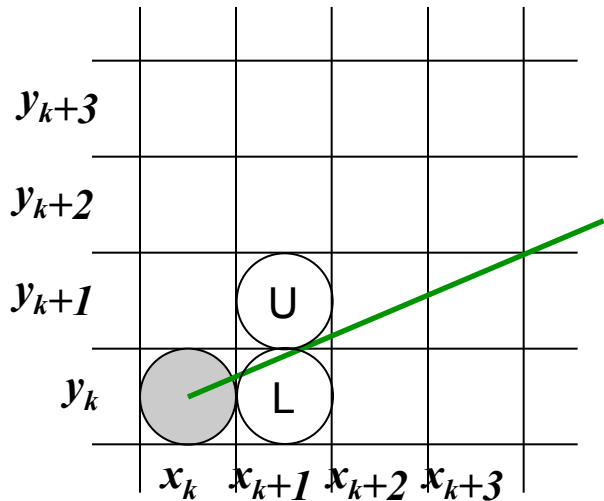
# Bresenham's Line Algorithm

- As  $m = \Delta y / \Delta x$ , our result becomes

$$\begin{aligned}d_{lower} - d_{upper} &= 2m(x_k + 1) - 2y_k + 2b - 1 \\&= 2 \frac{\Delta y}{\Delta x} (x_k + 1) - 2y_k + 2b - 1\end{aligned}$$

Uses only  
integers

- Define a *decision parameter* named  $p$  as:



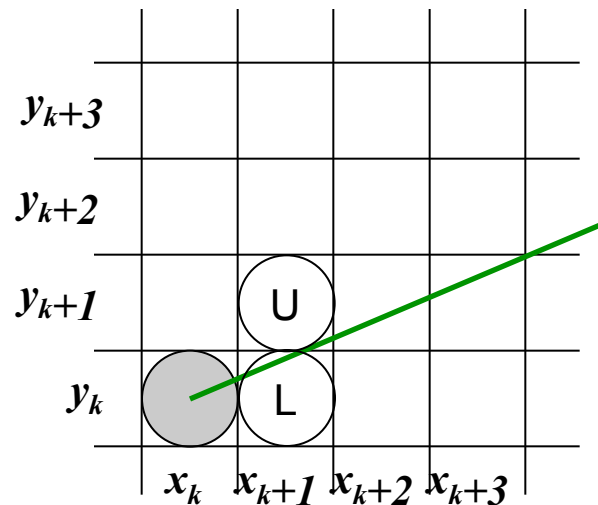
$$\begin{aligned}p_k &= \Delta x(d_{lower} - d_{upper}) \\&= 2\Delta y \cdot x_k + 2\Delta y - 2\Delta x \cdot y_k + \Delta x(2b - 1) \\&= 2\Delta y \cdot x_k - 2\Delta x \cdot y_k + c\end{aligned}$$

$$c = 2\Delta y + \Delta x(2b - 1)$$

$c$  is a constant  
independent of  
pixel position

# Bresenham's Line Algorithm

- Remember that  $p_k = \Delta x \cdot (d_{lower} - d_{upper})$
- This tells us which pixel is closer
  - If  $p_k > 0$ ,  $d_{lower} > d_{upper}$ , so U is closer
  - If  $p_k < 0$ ,  $d_{lower} < d_{upper}$ , so L is closer



# Bresenham's Line Algorithm

- The current decision variable:

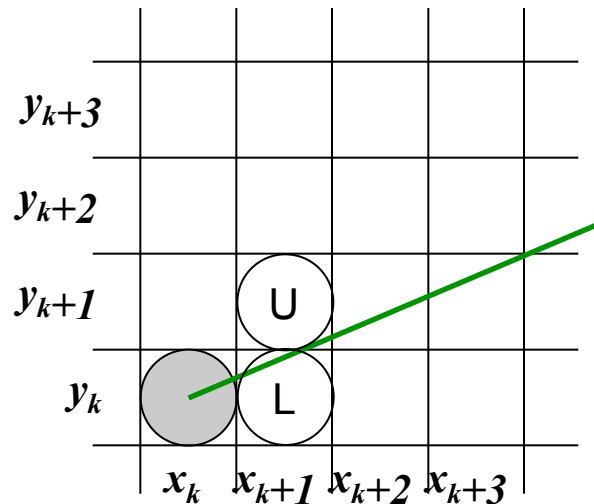
$$p_k = 2\Delta y \cdot x_k - 2\Delta x \cdot y_k + c$$

- The next decision variable:

$$p_{k+1} = 2\Delta y \cdot x_{k+1} - 2\Delta x \cdot y_{k+1} + c$$

- Difference (change from  $p_k$  to  $p_{k+1}$ ):

$$p_{k+1} - p_k = 2\Delta y(x_{k+1} - x_k) - 2\Delta x(y_{k+1} - y_k)$$



# Bresenham's Line Algorithm

- Change from  $p_k$  to  $p_{k+1}$  is  $2\Delta y(x_{k+1} - x_k) - 2\Delta x(y_{k+1} - y_k)$
- We know that  $x_{k+1} = x_k + 1$
- Thus, the next decision variable is:

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x(y_{k+1} - y_k)$$

- If  $p_k > 0$ ,  $y_{k+1} = y_k + 1$ , so  $p_{k+1} = p_k + 2\Delta y - 2\Delta x$
- If  $p_k < 0$ ,  $y_{k+1} = y_k$ , so  $p_{k+1} = p_k + 2\Delta y$

Recursive  
definition of  
the decision  
parameter

- What's left?

# Bresenham's Line Algorithm

---

- Must determine initial decision value
- First point plotted is  $(x_0, y_0)$ , so

$$p_0 = 2\Delta y \cdot x_0 - 2\Delta x \cdot y_0 + 2\Delta y + \Delta x(2b - 1)$$

- But  $(x_0, y_0)$  is exactly on the line, so

$$y_0 = mx_0 + b = \frac{\Delta y}{\Delta x} x_0 + b$$

- Substituting for  $y_0$ ,

$$\begin{aligned} p_0 &= 2\Delta y \cdot x_0 - 2\Delta x \left( \frac{\Delta y}{\Delta x} x_0 + b \right) + 2\Delta y + \Delta x(2b - 1) \\ &= 2\Delta y \cdot x_0 - 2\Delta y \cdot x_0 - 2b\Delta x + 2\Delta y + 2b\Delta x - \Delta x \\ &= 2\Delta y - \Delta x \end{aligned}$$

## Summary of Bresenham's Line Algorithm for $|m| < 1.0$

1. Input two line endpoints and store left endpoint in  $(x_0, y_0)$
2. Set the color for frame-buffer position  $(x_0, y_0)$  i.e., plot first point
3. Calculate the constants  $\Delta x, \Delta y, 2\Delta y$  and  $2\Delta y - 2\Delta x$ , and obtain the starting value for the decision parameter as:

$$p_0 = 2\Delta y - \Delta x$$

4. At each  $x_k$  along line, starting at  $k = 0$ , perform following test:  
If  $p_k < 0$ , the next point to plot is  $(x_k + 1, y_k)$  and

$$p_{k+1} = p_k + 2\Delta y$$

Otherwise, the next point to plot is  $(x_k + 1, y_k + 1)$  and

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

5. Repeat step 4  $\Delta x - 1$  times

# Bresenham's Line Algorithm

---

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
/* Bresenham line-drawing procedure for  $0 < m < 1$  */
```

```
void lineBres( int x0, int y0, int x1, int y1 ) {
```

```
    int dx = fabs(x1 - x0), dy = fabs(y1 - y0);
```

```
    int p = 2 * dy - dx;
```

```
    int twoDy = 2 * dy, twoDyMinusDx = 2 * (dy - dx);
```

```
    int x, y;
```

```
    /* Determine which endpoint to use as start position. */
```

```
    if( x0 > x1 ) {
```

```
        x = x1; /* draw (x1,y1) to (x0,y0) */
```

```
        y = y1;
```

```
        x1 = x0; /* use x1 as the ending x coordinate */
```

```
    } else {
```

```
        x = x0; /* draw (x0,y0) to (x1,y1) */
```

```
        y = y0;
```

```
    } /* continued on next slide */
```

# Bresenham's Line Algorithm

---

/\* continued from previous slide \*/

```
    setPixel( x, y );

    while( x < x1 ) {
        x++;
        if( p < 0 ) {
            p += twoDy;
        } else {
            y++;
            p += twoDyMinusDx;
        }
        setPixel( x, y );
    }
}
```



# Bresenham's Line Algorithm

---

- The algorithm is generalized to lines of arbitrary slope by considering the symmetry between the various octants and quadrants of the  $xy$ -plane.
- For example, a line with positive slope greater than 1.0
  - Interchange the roles of  $x$  and  $y$  directions
  - Step along  $y$  direction and calculate successive  $x$  values nearest to the line path
- Special cases such as horizontal or vertical lines can be loaded directly into the frame buffer without using the line algorithm.
- Performing scan conversion on polylines can be accomplished by running the algorithm  $n-1$  times to display the lines connecting the  $n$  endpoints.

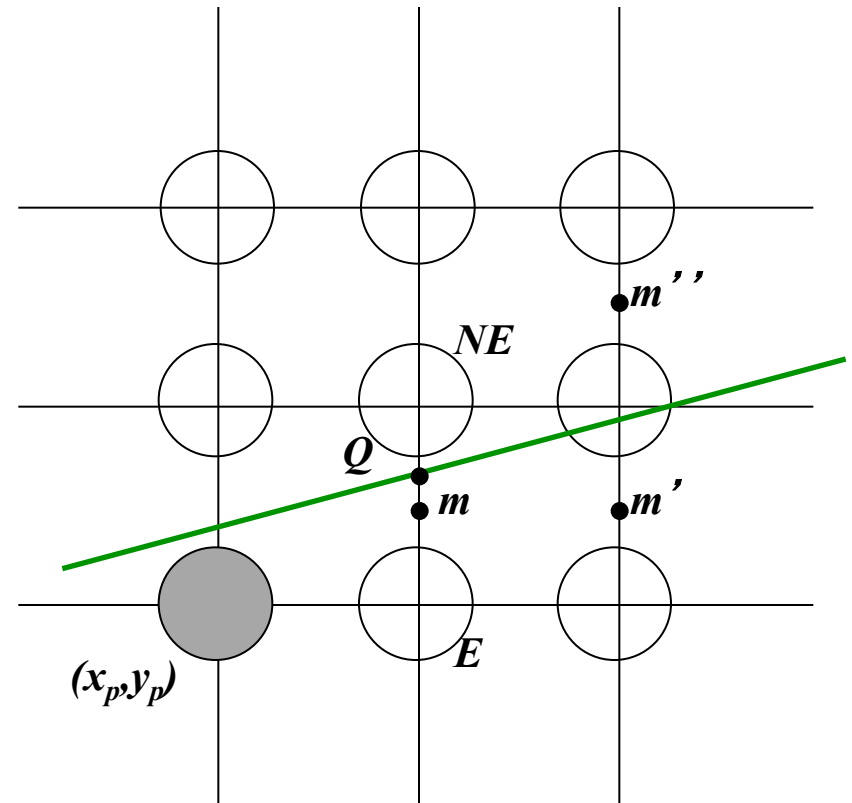
# Midpoint Line Algorithm

---

- Many implementations based on Bresenham's algorithm
- One version is the *midpoint line algorithm*
  - More easily extensible to other curves
- For simplicity, we'll assume:
  - Line is drawn left-to-right
    - Assumes  $p1$  is to the left of  $p2$
    - If not, can just swap the points
  - Slope of the line is between 0 and 1
    - Can deal with other slopes by reflecting around the axes

# Basic Concepts

- Consider this line segment
- Pixel at  $(x_p, y_p)$  already chosen
- Line intersects  $x_p + 1$  at  $Q$
- Bresenham's algorithm computes the distance from each pixel to  $Q$ , and selects the closest one
- Midpoint determines which side of the line  $m$  is on, and selects the appropriate pixel
- In this case, we select  $NE$



# Line Equations

---

- Recall these equations describing lines:

$$F(x, y) = ax + by + c = 0 \quad y = mx + B = \frac{dy}{dx}x + B$$

- Manipulating the slope/intercept form, we get:

$$y = \frac{dy}{dx}x + B$$

$$dx \cdot y = dy \cdot x + dx \cdot B$$

$$0 = dy \cdot x - dx \cdot y + dx \cdot B$$

$$F(x, y) = dy \cdot x - dx \cdot y + dx \cdot B$$

- From this, we get:  $a = dy$

$$b = -dx$$

$$c = dx \cdot B$$

# Line Equations

---

- We know that  $F(x,y)$  is
  - $= 0$  for  $(x,y)$  on the line
  - $< 0$  for  $(x,y)$  above the line
  - $> 0$  for  $(x,y)$  below the line
- Expensive to recompute  $F(x,y)$  each time
- Better to figure out how to compute it incrementally

# Line Equations

---

- For the midpoint  $m = (x_p + 1, y_p + 1/2)$ , we know that

$$F(m) = F(x_p + 1, y_p + 1/2)$$

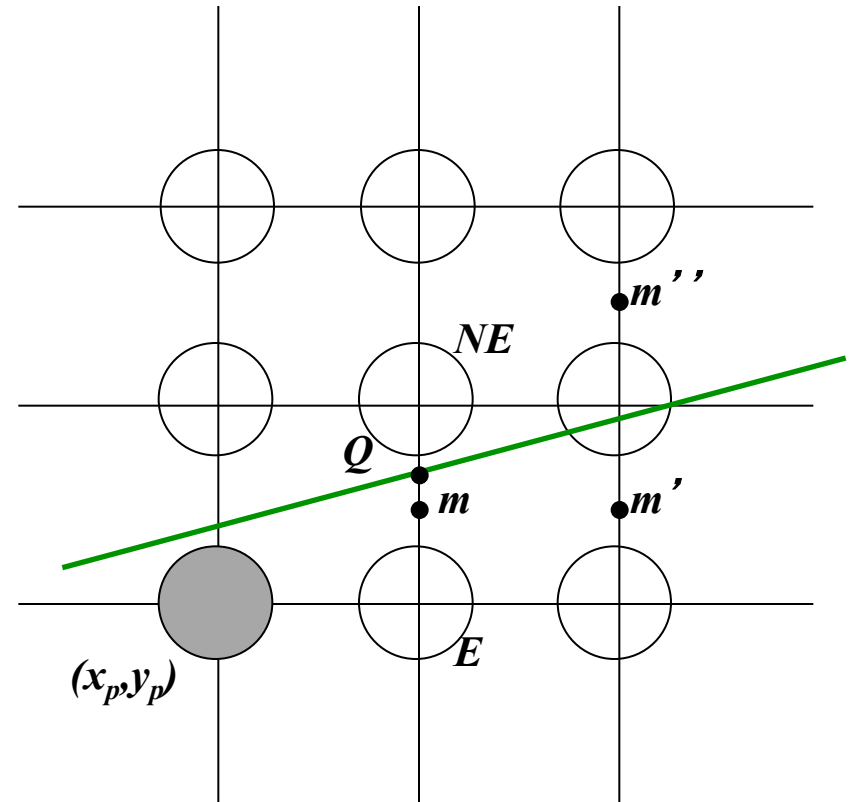
- We compute this as

$$d_{old} = F(x_p + 1, y_p + 1/2) = a(x_p + 1) + b(y_p + 1/2) + c$$

- If this is  $> 0$ , we'll choose  $NE$ ; if  $< 0$ , we'll choose  $E$

## The Next Level

- Now we know which way to move
- What's the next choice?
- If we chose E, new midpoint is  $m' = (x_p + 2, y_p + 1/2)$
- If we chose NE, new midpoint is  $m'' = (x_p + 2, y_p + 3/2)$
- From these, we'll calculate increments to be applied when we move in each direction



## Midpoint Deltas

---

- If we chose  $E$ , the new  $d$  is

$$d_{new} = F(x_p + 2, y_p + 1/2) = a(x_p + 2) + b(y_p + 1/2) + c$$

- We calculate  $\Delta E = d_{new} - d_{old}$ :

$$\begin{aligned}\Delta E &= F(m') - F(m) \\ &= (ax_p + 2a + by_p + b/2 + c) - (ax_p + a + by_p + b/2 + c) \\ &= ax_p - ax_p + 2a - a + by_p - by_p + b/2 - b/2 + c - c \\ &= a \\ &= dy\end{aligned}$$



## Midpoint Deltas

---

- If we chose  $NE$ , the new  $d$  is

$$d_{new} = F(x_p + 2, y_p + 3/2) = a(x_p + 2) + b(y_p + 3/2) + c$$

- We calculate  $\Delta NE = d_{new} - d_{old}$ :

$$\begin{aligned}\Delta NE &= F(m'') - F(m) \\ &= (ax_p + 2a + by_p + 3b/2 + c) - (ax_p + a + by_p + b/2 + c) \\ &= ax_p - ax_p + 2a - a + by_p - by_p + 3b/2 - b/2 + c - c \\ &= a + 2b/2 \\ &= a + b \\ &= dy - dx\end{aligned}$$

# Midpoint Deltas

---

- Set initial  $d$  to  $F(m)$ 
  - Modify by adding the appropriate delta value each time, depending on the pixel we selected
- For the initial midpoint at  $(x_0+1, y_0+1/2)$ ,  $F(m)$  is

$$\begin{aligned}F(m) &= F(x_0 + 1, y_0 + 1/2) \\&= a(x_0 + 1) + b(y_0 + 1/2) + c \\&= ax_0 + by_0 + c + a + b/2 \\&= F(x_0, y_0) + a + b/2 \\&= a + b/2 \\&= dy - dx/2\end{aligned}$$

- So, the initial decision value is  $dy - dx/2$

## Simplification

---

- Problem: calculating  $dy - dx/2$  requires division
- Solution: go back to  $F$ , and multiply everything by 2:

$$F(x, y) = 0 = ax + by + c$$

$$2 \times F(x, y) = 0 = 2(ax + by + c) = 2ax + 2by + 2c$$

- Carry this through our calculations
- $\Delta E = 2dy$
- $\Delta NE = 2(dy - dx)$
- Initial decision value =  $2dy - dx$

# Implementation

---

- Deltas are the increments when we choose  $E$  vs.  $NE$
- At each point, use  $d$  to determine direction to move
  - If  $d < 0$ , choose  $E$ 
    - Increment  $d$  by  $\Delta E$  ( $2dy$ )
  - If  $d > 0$ , choose  $NE$ 
    - Increment  $y$  by one
    - Increment  $d$  by  $\Delta NE$  ( $2(dy - dx)$ )
- What if  $d = 0$ ?
  - Pick one or the other – equally close
  - We'll choose  $E$

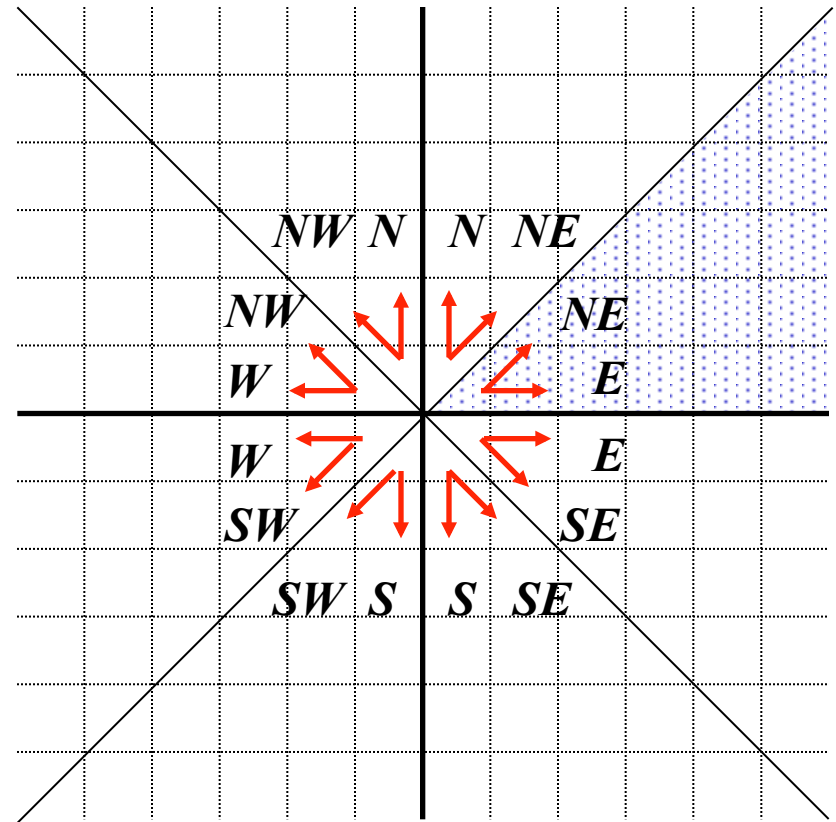
# Implementation

---

```
void midpointLine( int x0, int y0, int x1, int y1) {  
    int dE, dNE, x, y, d;  
    int dy = y1 - y0, dx = x1 - x0;  
  
    dE = 2 * dy;  
    dNE = 2 * ( dy - dx );  
    d = dE - dx; /* 2*dy - dx */  
  
    for( x = x0, y = y0; x <= x1; ++x ) {  
        setPixel( x, y );  
  
        if( d <= 0 ) { /* choose E */  
            d += dE;  
        } else { /* choose NE */  
            ++y;  
            d += dNE;  
        }  
    }  
}
```

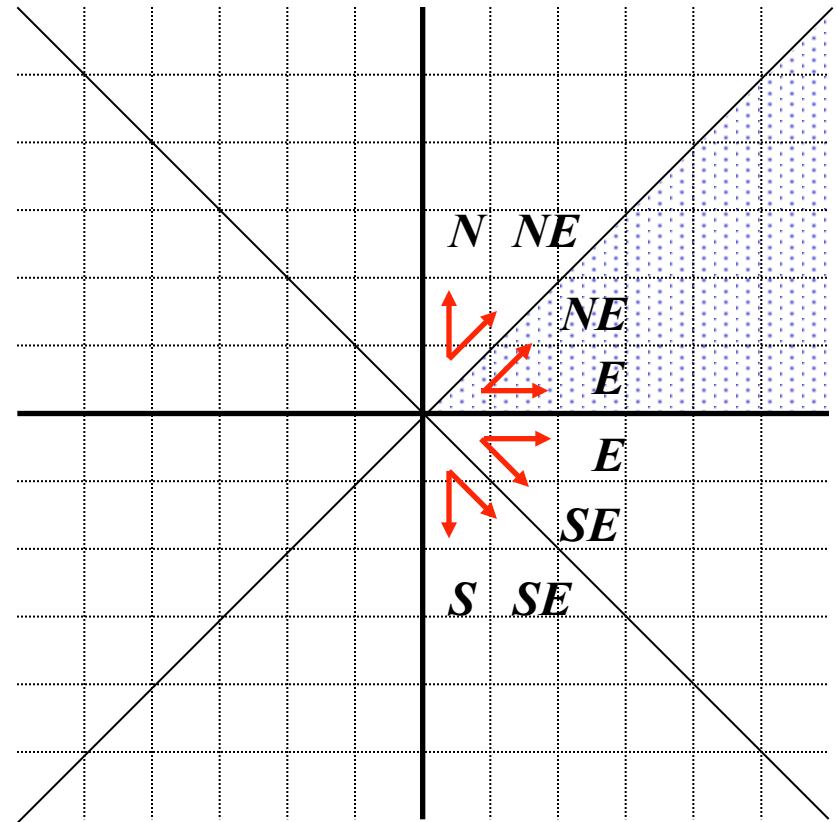
# Extending the Midpoint Line Algorithm

- We considered  $0 < m < 1$
- Can get significant performance improvement by looking for special cases
  - $m = 0$
  - $m = 1$
  - $m = \infty$
- For other slopes, merely choose  $m, m', m''$ , and directions differently
- For large slopes, loop on  $y$  rather than  $x$



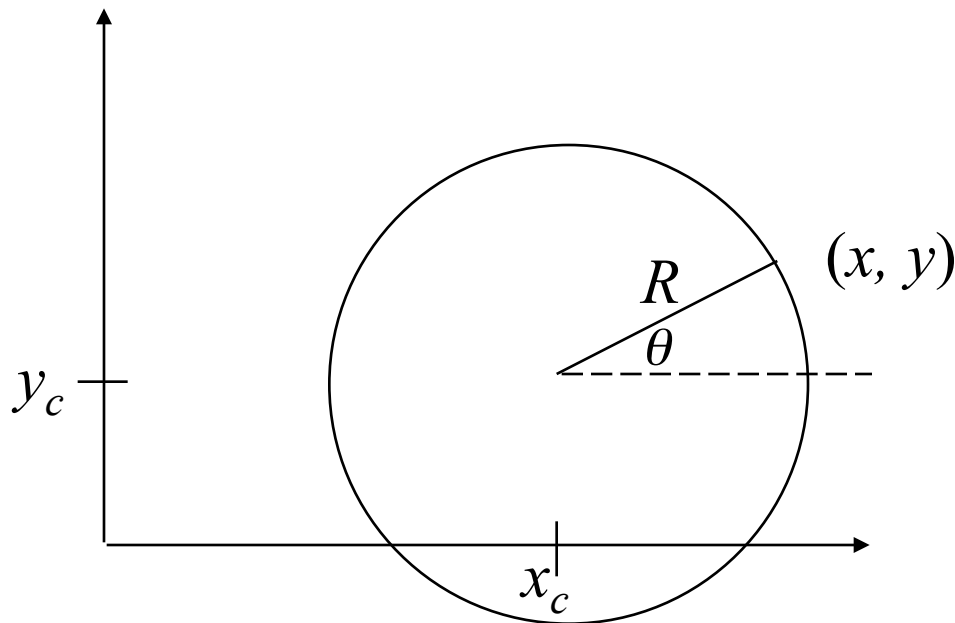
# Extending the Midpoint Line Algorithm

- Lines in the left half are right-to-left lines
- If we swap the starting and ending points, they become left-to-right lines
- Thus, only really need to consider L-R line segments
- However, this has implications for line styles other than solid



# Drawing Circles

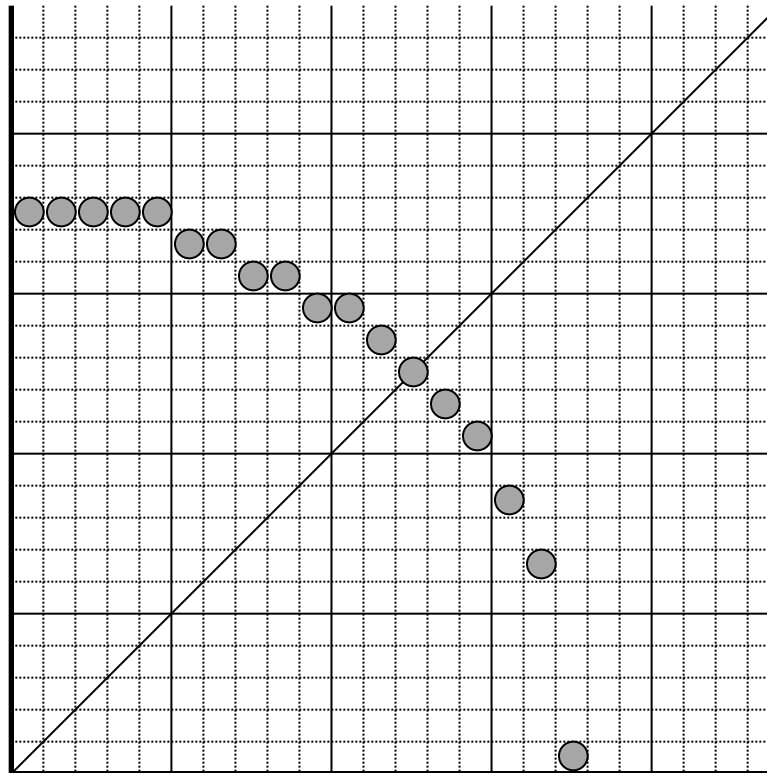
- We describe a circle with  $(x - x_c)^2 + (y - y_c)^2 = R^2$
- Solving for y, we get:  $y = y_c \pm \sqrt{R^2 - (x_c - x)^2}$
- This involves considerable computation





# Drawing Circles

- Stepping along the x axis and plotting corresponding y values results in non uniform spaces between pixel positions



# Drawing Circles

---

- In general, as  $x$  approaches  $R$  (except for large  $R$ ), we'll get gaps in the circle

- Get better results by using parametric polar form

$$x = x_c + R \cos \theta$$

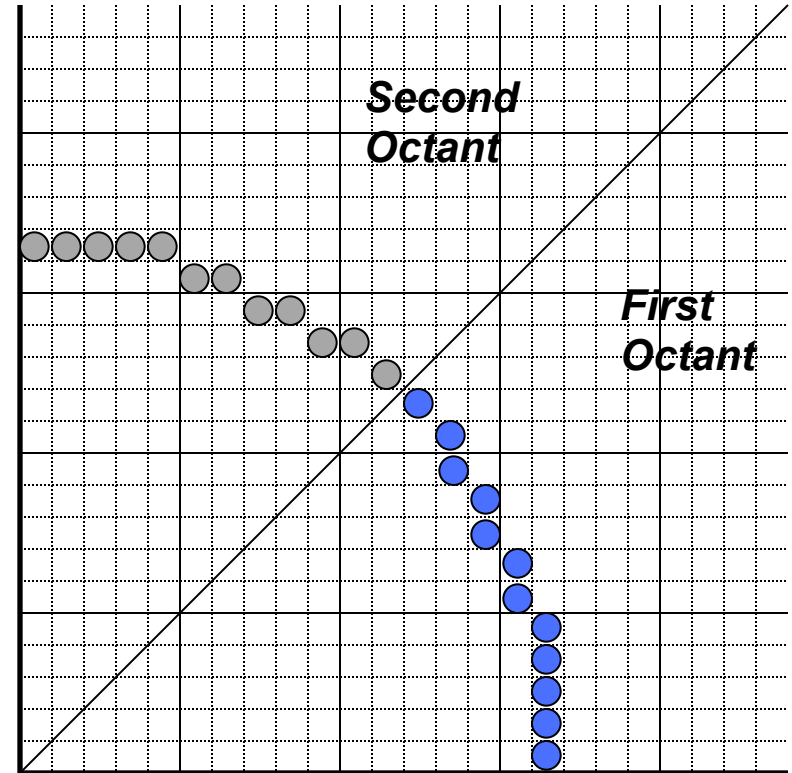
$$y = y_c + R \sin \theta$$

- and plotting  $(R \cos \theta, R \sin \theta)$  for  $\theta$  from  $0^\circ$  to  $90^\circ$
- However, this is still very inefficient
  - Lots of floating-point and trigonometric computations
- How can we get better results in less time?

# Eight-Way Symmetry

- We can use the symmetry of a circle to improve efficiency
- Once first octant is computed:
- We can mirror this across the  $x$  and  $y$  axes, and across both  $45^\circ$  lines
- Mirroring is trivial:
- First quadrant:  $(x,y)$   $(y,x)$
- Second quadrant:  $(y,-x)$   $(x,-y)$
- Third quadrant:  $(-x,-y)$   $(-y,-x)$
- Fourth quadrant:  $(-y,x)$   $(-x,y)$

Still have square roots or trigonometric computations for first octant



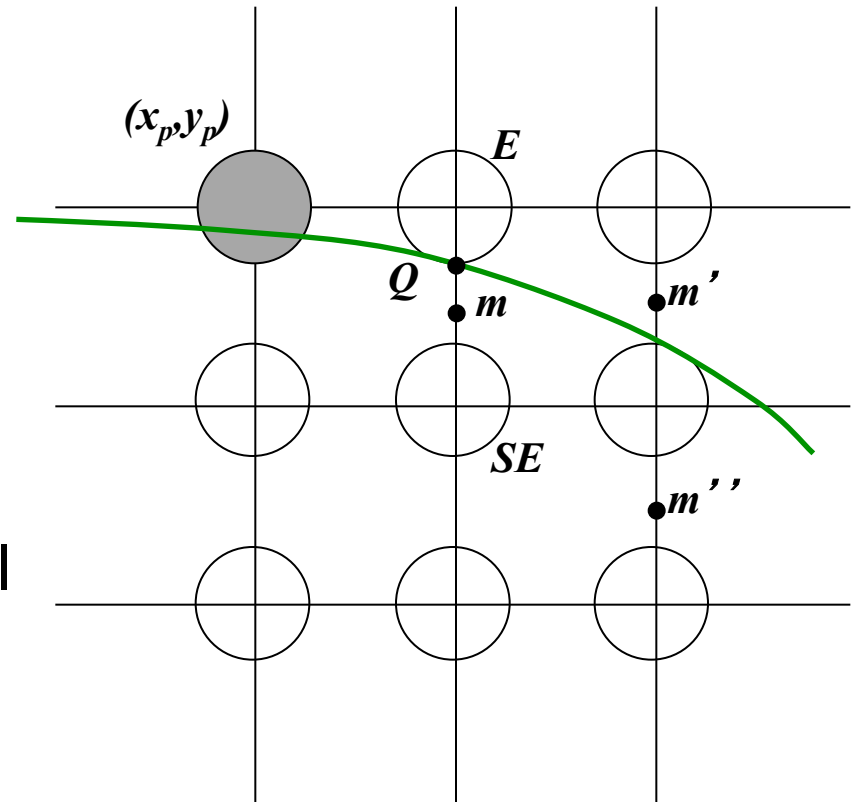
# Midpoint Circle Algorithm

---

- Bresenham developed an incremental algorithm for circle generation, too
  - Originally designed for use with pen plotters
  - Generates all points around entire circumference
- We can combine this with our 8-way symmetry approach to do integer incremental circle drawing
- We'll look only at the first octant. Symmetry can be used to generate rest of circle.

# Basic Concepts

- Pixel at  $(x_p, y_p)$  already chosen
- Must choose the next pixel
- Curve intersects for  $x_p + 1$  at point  $Q$
- Midpoint determines which side of the line  $m$  is on, and selects the appropriate pixel
- In this case, we select  $E$



# Circle Equations

---

- Recall the basic circle equation (centered at origin):

$$F(x,y) = x^2 + y^2 - R^2$$

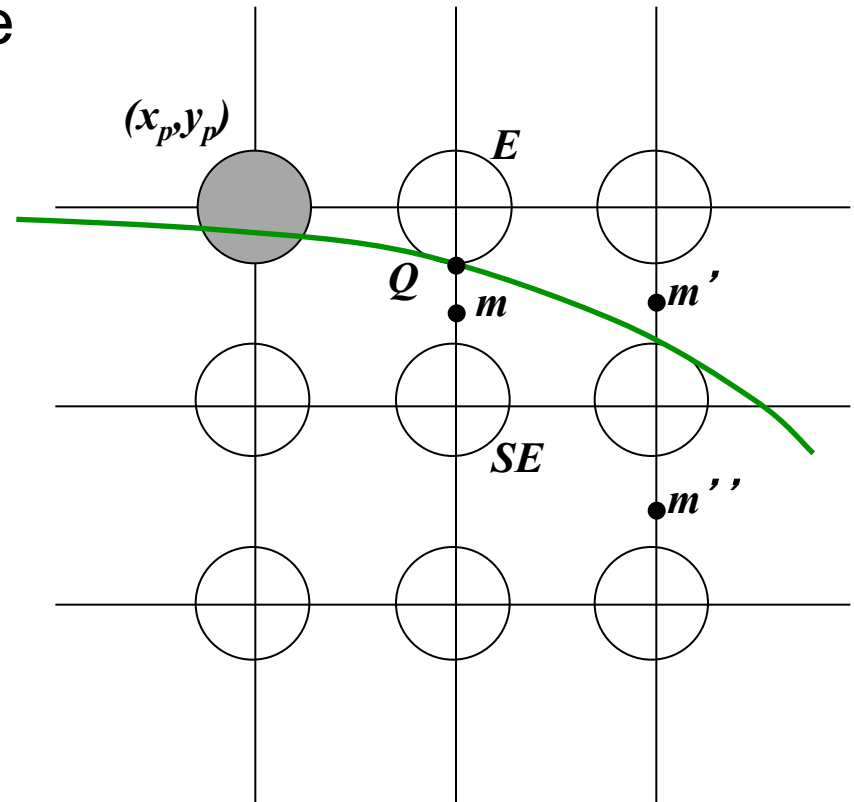
- $F(x,y)$  is zero on the circle, negative inside the circle, and positive outside the circle
- As before, want to compute  $F(x,y)$  incrementally
- For the midpoint  $m$ , we compute

$$\begin{aligned} d_{old} &= F(x_p + 1, y_p - 1/2) \\ &= (x_p + 1)^2 + (y_p - 1/2)^2 - R^2 \end{aligned}$$

- If this is  $< 0$ , we'll choose  $E$ ; if  $> 0$ , we'll choose  $SE$

## The Next Level

- Now, look at the next choice
- If we choose  $E$ , the new midpoint is  $m'$
- If we choose  $SE$ , the new midpoint is  $m''$



## Midpoint Deltas

---

- If we choose  $E$ , the new  $d$  is

$$\begin{aligned}d_{new} &= F(x_p + 2, y_p - 1/2) \\ &= (x_p + 2)^2 + (y_p - 1/2)^2 - R^2\end{aligned}$$

- We can calculate  $d_{new} - d_{old}$  to get  $\Delta E = 2x_p + 3$



## Midpoint Deltas

---

- If we choose  $SE$ , the new  $d$  is

$$\begin{aligned}d_{new} &= F(x_p + 2, y_p - 3/2) \\ &= (x_p + 2)^2 + (y_p - 3/2)^2 - R^2\end{aligned}$$

- And we calculate  $d_{new} - d_{old}$  to get  $\Delta E = 2x_p - 2y_p + 5$
- Note that the deltas vary according to the position of the previous pixel chosen at  $(x_p, y_p)$

# First Implementation

- We know that the starting pixel is at  $(0, R)$
- The first midpoint is at  $(1, R - 1/2)$ , so

$$\begin{aligned} F(m) &= (0 + 1)^2 + (R - 1/2)^2 - R^2 \\ &= 1 + (R^2 - R + 1/4) - R^2 \\ &= 5/4 - R \end{aligned}$$

- Implementation:

```
void midpointCircle(int radius) {  
    int x, y; float d;  
  
    x = 0;  
    y = radius;  
    d = 5.0 / 4 - radius;  
  
    drawAllEightPoints( x, y );  
  
    while( y > x ) {  
        if( d < 0 ) {  
            d += x * 2.0 + 3;  
            ++x;  
        } else {  
            d += (x - y) * 2.0 + 5;  
            ++x;  
            --y;  
        }  
  
        drawAllEightPoints( x, y );  
    }  
}
```

# Simplification

---

- Problem: requires floating point arithmetic
- Solution: transform the program
- Define a new decision variable,  $h$ , as  $h = d - \frac{1}{4}$
- Substitute  $h + \frac{1}{4}$  for  $d$  in the code
- Initialization is now  $h = 1 - R$ , and the comparison  $d < 0$  becomes  $h < -\frac{1}{4}$
- However:
  - $h$  starts out with an integer value
  - Is always incremented by integer values
- Thus, comparison can be simplified to  $h < 0$

# Integer Implementation

---

```
void midpointCircle( int radius ) {  
    int x, y, h;  
  
    x = 0;  
    y = radius;  
    h = 1 - radius;  
  
    drawAllEightPoints( x, y );  
  
    while( y > x ) {  
        if( h < 0 ) { /* choose E */  
            h += x * 2 + 3;  
            ++x;  
        } else { /* choose SE */  
            h += (x - y) * 2 + 5;  
            ++x;  
            --y;  
        }  
  
        drawAllEightPoints( x, y );  
    }  
}
```

## Further Simplification

---

- We can further improve the algorithm's performance by using *second-order differences*
- We noted that the deltas are linear equations, and computed them directly
- We can apply the same technique to allow us to compute the deltas incrementally

## Further Simplification

---

- If we choose the  $E$  pixel,  $\Delta E = 2x_p + 3$ ; call this  $\Delta E_{old}$
- Calculate  $\Delta E_{new}$  at  $(x_p + 1, y_p)$  as  $2(x_p + 1) + 3$
- Similarly, we calculate  $\Delta SE_{new} - \Delta SE_{old} = 2$
- From this,  $\Delta E_{new} - \Delta E_{old} = 2$
- If we choose  $SE$  instead of  $E$ , the deltas are 2 and 4, respectively

## Second-Order Integer Implementation

---

```
void midpointCircle2( int radius ) {  
    int x, y, h, dE, dSE;  
  
    x = 0;  y = radius;  h = 1 - radius;  
    dE = 3;  dSE = 5 - radius * 2; // initial deltas  
    drawAllEightPoints( x, y );  
    while( y > x ) {  
        if( h < 0 ) {  
            h += dE;  dE += 2;  dSE += 2;  
            ++x;  
        } else {  
            h += dSE;  dE += 2;  dSE += 4;  
            ++x;  
            --y;  
        }  
        drawAllEightPoints( x, y );  
    }  
}
```

## Other Curved Shapes

---

- Ellipse
- Calculated from two foci,  $F1$  and  $F2$
- Sum  $d1 + d2$  is constant

